In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, MinMaxScaler, StandardScaler

from sklearn.tree import DecisionTreeRegressor
from sklearn.neighbors import KNeighborsRegressor

from sklearn.metrics import root_mean_squared_error

from sklearn.model_selection import cross_val_score
from sklearn.model_selection import GridSearchCV

from sklearn.pipeline import make_pipeline

!pip install ydata-profiling
from ydata_profiling import ProfileReport
```

```
Requirement already satisfied: ydata-profiling in c:\users\ayabo\anaconda3\lib\site-packages (4.18.1)
Requirement already satisfied: scipy<1.17,>=1.8 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (1.15.3)
Requirement already satisfied: pandas!=1.4.0,<3.0,>1.5 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (2.2.3)
Requirement already satisfied: matplotlib<=3.10,>=3.5 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (3.10.0)
Requirement already satisfied: pydantic<3,>=2 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (2.10.3)
Requirement already satisfied: PyYAML<6.1,>=6.0.3 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (6.0.3)
Requirement already satisfied: jinja2<3.2,>=3.1.6 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (3.1.6)
Requirement already satisfied: visions<0.8.2,>=0.7.5 in c:\users\ayabo\anaconda3\lib\site-packages (from visions[type_image_path]<0.8.2,>=
0.7.5->ydata-profiling) (0.8.1)
Requirement already satisfied: numpy<2.4,>=1.22 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (2.1.3)
Requirement already satisfied: minify-html>=0.15.0 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (0.18.1)
Requirement already satisfied: filetype>=1.0.0 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (1.2.0)
Requirement already satisfied: phik<0.13,>=0.12.5 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (0.12.5)
Requirement already satisfied: requests<3,>=2.32.0 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (2.32.3)
Requirement already satisfied: tqdm<5,>=4.66.3 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (4.67.1)
Requirement already satisfied: seaborn<0.14,>=0.10.1 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (0.13.2)
Requirement already satisfied: multimethod<2,>=1.4 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (1.12)
Requirement already satisfied: statsmodels<1,>=0.13.2 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (0.14.4)
Requirement already satisfied: typeguard<5,>=4 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (4.5.1)
Requirement already satisfied: imagehash==4.3.2 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (4.3.2)
Requirement already satisfied: wordcloud>=1.9.4 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (1.9.6)
Requirement already satisfied: dacite<2,>=1.9 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (1.9.2)
Requirement already satisfied: numba<0.63,>=0.60 in c:\users\ayabo\anaconda3\lib\site-packages (from ydata-profiling) (0.61.0)
Requirement already satisfied: PyWavelets in c:\users\ayabo\anaconda3\lib\site-packages (from imagehash==4.3.2->ydata-profiling) (1.8.0)
Requirement already satisfied: pillow in c:\users\ayabo\anaconda3\lib\site-packages (from imagehash==4.3.2->ydata-profiling) (11.1.0)
Requirement already satisfied: MarkupSafe>=2.0 in c:\users\ayabo\anaconda3\lib\site-packages (from jinja2<3.2,>=3.1.6->ydata-profiling) (3.
0.2)
Requirement already satisfied: contourpy>=1.0.1 in c:\users\ayabo\anaconda3\lib\site-packages (from matplotlib<=3.10,>=3.5->ydata-profilin
g) (1.3.1)
Requirement already satisfied: cycler>=0.10 in c:\users\ayabo\anaconda3\lib\site-packages (from matplotlib<=3.10,>=3.5->ydata-profiling)
(0.11.0)
Requirement already satisfied: fonttools>=4.22.0 in c:\users\ayabo\anaconda3\lib\site-packages (from matplotlib<=3.10,>=3.5->ydata-profilin
g) (4.55.3)
Requirement already satisfied: kiwisolver>=1.3.1 in c:\users\ayabo\anaconda3\lib\site-packages (from matplotlib<=3.10,>=3.5->ydata-profilin
g) (1.4.8)
Requirement already satisfied: packaging>=20.0 in c:\users\ayabo\anaconda3\lib\site-packages (from matplotlib<=3.10,>=3.5->ydata-profiling)
(24.2)
Requirement already satisfied: pyparsing>=2.3.1 in c:\users\ayabo\anaconda3\lib\site-packages (from matplotlib<=3.10,>=3.5->ydata-profilin
g) (3.2.0)
Requirement already satisfied: python-dateutil>=2.7 in c:\users\ayabo\anaconda3\lib\site-packages (from matplotlib<=3.10,>=3.5->ydata-profi
ling) (2.9.0.post0)
```

```
Requirement already satisfied: llvmlite<0.45,>=0.44.0dev0 in c:\users\ayabo\anaconda3\lib\site-packages (from numba<0.63,>=0.60->ydata-prof
iling) (0.44.0)
Requirement already satisfied: pytz>=2020.1 in c:\users\ayabo\anaconda3\lib\site-packages (from pandas!=1.4.0,<3.0,>1.5->ydata-profiling)
(2024.1)
Requirement already satisfied: tzdata>=2022.7 in c:\users\ayabo\anaconda3\lib\site-packages (from pandas!=1.4.0,<3.0,>1.5->ydata-profiling)
(2025.2)
Requirement already satisfied: joblib>=0.14.1 in c:\users\ayabo\anaconda3\lib\site-packages (from phik<0.13,>=0.12.5->ydata-profiling) (1.
4.2)
Requirement already satisfied: annotated-types>=0.6.0 in c:\users\ayabo\anaconda3\lib\site-packages (from pydantic<3,>=2->ydata-profiling)
(0.6.0)
Requirement already satisfied: pydantic-core==2.27.1 in c:\users\ayabo\anaconda3\lib\site-packages (from pydantic<3,>=2->ydata-profiling)
(2.27.1)
Requirement already satisfied: typing-extensions>=4.12.2 in c:\users\ayabo\anaconda3\lib\site-packages (from pydantic<3,>=2->ydata-profilin
g) (4.15.0)
Requirement already satisfied: charset-normalizer<4,>=2 in c:\users\ayabo\anaconda3\lib\site-packages (from requests<3,>=2.32.0->ydata-prof
iling) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in c:\users\ayabo\anaconda3\lib\site-packages (from requests<3,>=2.32.0->ydata-profiling) (3.7)
Requirement already satisfied: urllib3<3,>=1.21.1 in c:\users\ayabo\anaconda3\lib\site-packages (from requests<3,>=2.32.0->ydata-profiling)
(2.3.0)
Requirement already satisfied: certifi>=2017.4.17 in c:\users\ayabo\anaconda3\lib\site-packages (from requests<3,>=2.32.0->ydata-profiling)
(2025.4.26)
Requirement already satisfied: patsy>=0.5.6 in c:\users\ayabo\anaconda3\lib\site-packages (from statsmodels<1,>=0.13.2->ydata-profiling)
(1.0.1)
Requirement already satisfied: colorama in c:\users\ayabo\anaconda3\lib\site-packages (from tqdm<5,>=4.66.3->ydata-profiling) (0.4.6)
Requirement already satisfied: attrs>=19.3.0 in c:\users\ayabo\anaconda3\lib\site-packages (from visions<0.8.2,>=0.7.5->visions[type_image_
path]<0.8.2,>=0.7.5->ydata-profiling) (24.3.0)
Requirement already satisfied: networkx>=2.4 in c:\users\ayabo\anaconda3\lib\site-packages (from visions<0.8.2,>=0.7.5->visions[type_image_
path]<0.8.2,>=0.7.5->ydata-profiling) (3.4.2)
Requirement already satisfied: puremagic in c:\users\ayabo\anaconda3\lib\site-packages (from visions<0.8.2,>=0.7.5->visions[type_image_pat
h]<0.8.2,>=0.7.5->ydata-profiling) (2.0.2)
Requirement already satisfied: six>=1.5 in c:\users\ayabo\anaconda3\lib\site-packages (from python-dateutil>=2.7->matplotlib<=3.10,>=3.5->y
data-profiling) (1.17.0)
```

Import and Exploratory Data Analysis

In [2]:
```
df = pd.read_csv('SeoulBikeData.csv', encoding='windows-1252')

df.head()
```

Out[2]:

	Date	Rented Bike Count	Hour	Temperature(°C)	Humidity(%)	Wind speed (m/s)	Visibility (10m)	Dew point temperature(°C)	Solar Radiation (MJ/m2)	Rainfall(mm)	Snowfall (cm)	Seasons	Holiday	I
0	01/12/2017	254	0	-5.2	37	2.2	2000	-17.6	0.0	0.0	0.0	Winter	No Holiday	
1	01/12/2017	204	1	-5.5	38	0.8	2000	-17.6	0.0	0.0	0.0	Winter	No Holiday	
2	01/12/2017	173	2	-6.0	39	1.0	2000	-17.7	0.0	0.0	0.0	Winter	No Holiday	
3	01/12/2017	107	3	-6.2	40	0.9	2000	-17.6	0.0	0.0	0.0	Winter	No Holiday	
4	01/12/2017	78	4	-6.0	36	2.3	2000	-18.6	0.0	0.0	0.0	Winter	No Holiday	

In [3]:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8760 entries, 0 to 8759
Data columns (total 14 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   Date                       8760 non-null   object
 1   Rented Bike Count          8760 non-null   int64
 2   Hour                       8760 non-null   int64
 3   Temperature(°C)            8760 non-null   float64
 4   Humidity(%)                8760 non-null   int64
 5   Wind speed (m/s)           8760 non-null   float64
 6   Visibility (10m)           8760 non-null   int64
 7   Dew point temperature(°C)  8760 non-null   float64
 8   Solar Radiation (MJ/m2)    8760 non-null   float64
 9   Rainfall(mm)               8760 non-null   float64
 10  Snowfall (cm)              8760 non-null   float64
 11  Seasons                    8760 non-null   object
 12  Holiday                    8760 non-null   object
 13  Functioning Day            8760 non-null   object
dtypes: float64(6), int64(4), object(4)
memory usage: 958.3+ KB
```

In [4]:
```
df.describe(include='all')
```

Out[4]:

	Date	Rented Bike Count	Hour	Temperature(°C)	Humidity(%)	Wind speed (m/s)	Visibility (10m)	Dew point temperature(°C)	Solar Radiation (MJ/m2)	Rainfall(mm)	
count	8760	8760.000000	8760.000000	8760.000000	8760.000000	8760.000000	8760.000000	8760.000000	8760.000000	8760.000000	87
unique	365	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
top	30/11/2018	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
freq	24	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	NaN	
mean	NaN	704.602055	11.500000	12.882922	58.226256	1.724909	1436.825799	4.073813	0.569111	0.148687	
std	NaN	644.997468	6.922582	11.944825	20.362413	1.036300	608.298712	13.060369	0.868746	1.128193	
min	NaN	0.000000	0.000000	-17.800000	0.000000	0.000000	27.000000	-30.600000	0.000000	0.000000	
25%	NaN	191.000000	5.750000	3.500000	42.000000	0.900000	940.000000	-4.700000	0.000000	0.000000	
50%	NaN	504.500000	11.500000	13.700000	57.000000	1.500000	1698.000000	5.100000	0.010000	0.000000	
75%	NaN	1065.250000	17.250000	22.500000	74.000000	2.300000	2000.000000	14.800000	0.930000	0.000000	
max	NaN	3556.000000	23.000000	39.400000	98.000000	7.400000	2000.000000	27.200000	3.520000	35.000000	

In [5]:
```
#df.isna().sum()
```

In [6]:
```
profile = ProfileReport(df, explorative=True)
profile.to_notebook_iframe()
```

```
Summarize dataset:   0%|          | 0/5 [00:00<?, ?it/s]
%|                                                                                                    | 0/14 [00:00<?, ?it/s]
100%|└└└
```

YData Profiling Report

Overview

Overview | Alerts 12 | Reproduction

Dataset statistics

Number of variables	14
Number of observations	8760
Missing cells	0
Missing cells (%)	0.0%
Duplicate rows	0
Duplicate rows (%)	0.0%
Total size in memory	2.5 MiB
Average record size in memory	304.8 B

Variable types

DateTime	1
Numeric	10
Categorical	2
Boolean	1

Variables

Select Columns

Date

Profiling report flagged high correlations among multiple variables, so we check for multicolinearity using VIF. We try to determine and select which of these multi correlated variables is informative and critical for the model, to reduce redundancy and misleading interpretations, and improve model stability & performance.

Multicollinearity Check

N/B:

- VIF starts at 1 an has no upper limit
- VIF = 1 -- no correlation between an independent variable and other variables
- VIF > ~5 -- high multicollinearity between the independent variable and others

In [7]:
```
from statsmodels.stats.outliers_influence import variance_inflation_factor

# function to determine VIFs
def det_vif(X):
    vif = pd.DataFrame()
    vif['variables'] = X.columns
    vif['VIF'] = [variance_inflation_factor(X.values, i) for i in range (X.shape[1])]

    return (vif)
```

In [8]:
```
# target (Rented Bike Counts) dropped when determining VIFs
# Dew point temperature also dropped as it was flagged by the profile report for multiple high correlations, which may inflate VIF values
# non-numeric columns also dropped

det_vif(df.drop(['Dew point temperature(°C)','Rented Bike Count'], axis=1).select_dtypes(include='number'))
```

Out[8]:

	variables	VIF
0	Hour	3.921832
1	Temperature(°C)	3.228318
2	Humidity(%)	4.868221
3	Wind speed (m/s)	4.608625
4	Visibility (10m)	4.710170
5	Solar Radiation (MJ/m2)	2.246791
6	Rainfall(mm)	1.079158
7	Snowfall (cm)	1.120579

All numeric variables show moderate and acceptable VIF values (<5), indicating that each of these variables contributes unique and independent information to the model. Assessing multicollinearity via VIF is important for regression problems, as high variable correlations can distort model coefficients and reduce intepretability and reliability - since regression explicitly measures relationships between each variable and the target.

Feature Engineering and Selection

Dropping features

In [9]:

```
# drop dates (unique values), functioning day (boolean Yes/No) -- not informative
# and 'Dew point temperature(°C) since it has high correlations with multiple other vairables

data = df.drop(['Date', 'Functioning Day', 'Dew point temperature(°C)'], axis=1)
data.head()
```

Out[9]:

	Rented Bike Count	Hour	Temperature(°C)	Humidity(%)	Wind speed (m/s)	Visibility (10m)	Solar Radiation (MJ/m2)	Rainfall(mm)	Snowfall (cm)	Seasons	Holiday
0	254	0	-5.2	37	2.2	2000	0.0	0.0	0.0	Winter	No Holiday
1	204	1	-5.5	38	0.8	2000	0.0	0.0	0.0	Winter	No Holiday
2	173	2	-6.0	39	1.0	2000	0.0	0.0	0.0	Winter	No Holiday
3	107	3	-6.2	40	0.9	2000	0.0	0.0	0.0	Winter	No Holiday
4	78	4	-6.0	36	2.3	2000	0.0	0.0	0.0	Winter	No Holiday

Encoding categorical features

-- regression needs a numerical representation of categorical features

In [10]:
```
# One-hot encoding

seasons = pd.get_dummies(df['Seasons'], drop_first=True).astype(int)
holiday = pd.get_dummies(df['Holiday'], drop_first=True).astype(int)
```

In [11]:
```
data.drop(['Seasons', 'Holiday'], axis=1, inplace=True)

data = pd.concat([data, seasons, holiday], axis=1)
data.head()
```

Out[11]:

	Rented Bike Count	Hour	Temperature(°C)	Humidity(%)	Wind speed (m/s)	Visibility (10m)	Solar Radiation (MJ/m2)	Rainfall(mm)	Snowfall (cm)	Spring	Summer	Winter	No Holiday
0	254	0	-5.2	37	2.2	2000	0.0	0.0	0.0	0	0	1	1
1	204	1	-5.5	38	0.8	2000	0.0	0.0	0.0	0	0	1	1
2	173	2	-6.0	39	1.0	2000	0.0	0.0	0.0	0	0	1	1
3	107	3	-6.2	40	0.9	2000	0.0	0.0	0.0	0	0	1	1
4	78	4	-6.0	36	2.3	2000	0.0	0.0	0.0	0	0	1	1

In [12]:
```
data.info()
```

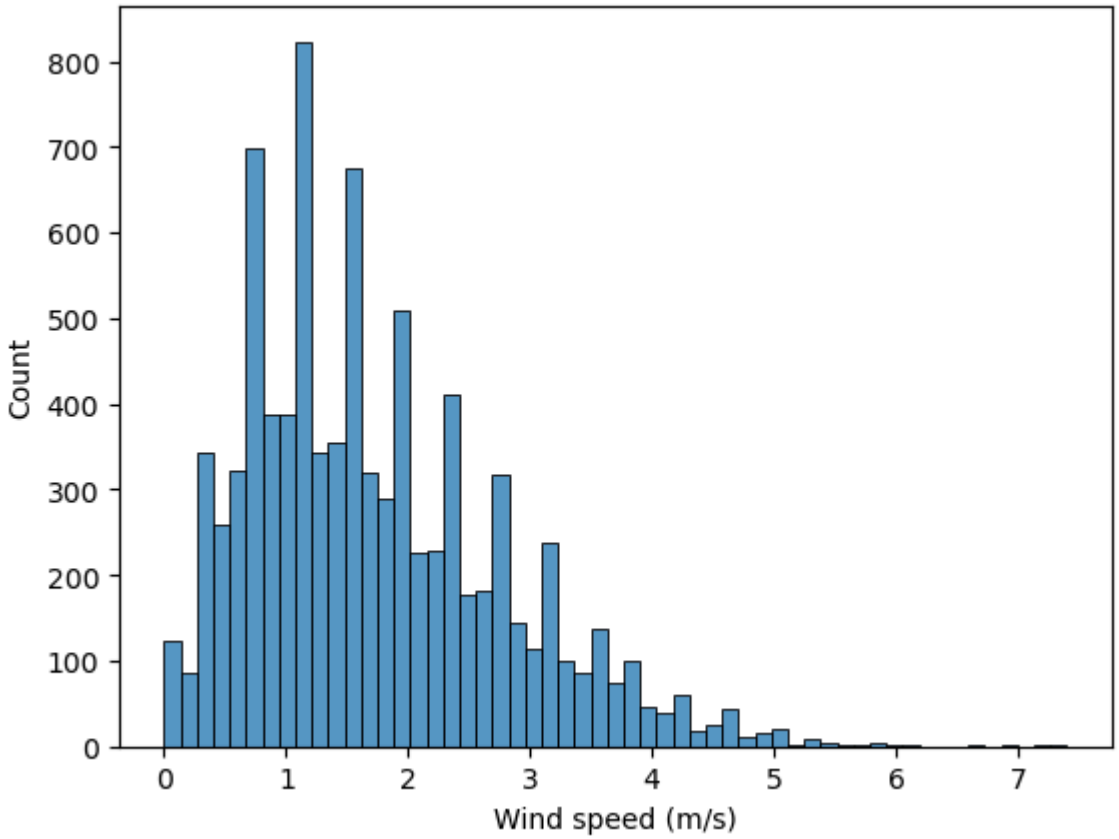
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 8760 entries, 0 to 8759
Data columns (total 13 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Rented Bike Count       8760 non-null   int64
 1   Hour                    8760 non-null   int64
 2   Temperature(°C)         8760 non-null   float64
 3   Humidity(%)             8760 non-null   int64
 4   Wind speed (m/s)        8760 non-null   float64
 5   Visibility (10m)        8760 non-null   int64
 6   Solar Radiation (MJ/m2) 8760 non-null   float64
 7   Rainfall(mm)            8760 non-null   float64
 8   Snowfall (cm)           8760 non-null   float64
 9   Spring                  8760 non-null   int64
 10  Summer                  8760 non-null   int64
 11  Winter                  8760 non-null   int64
 12  No Holiday              8760 non-null   int64
dtypes: float64(5), int64(8)
memory usage: 889.8 KB
```

Remove outliers

-- regression models are sensitive to outliers, which can disproportionately influence model's coefficients/weights and predictions

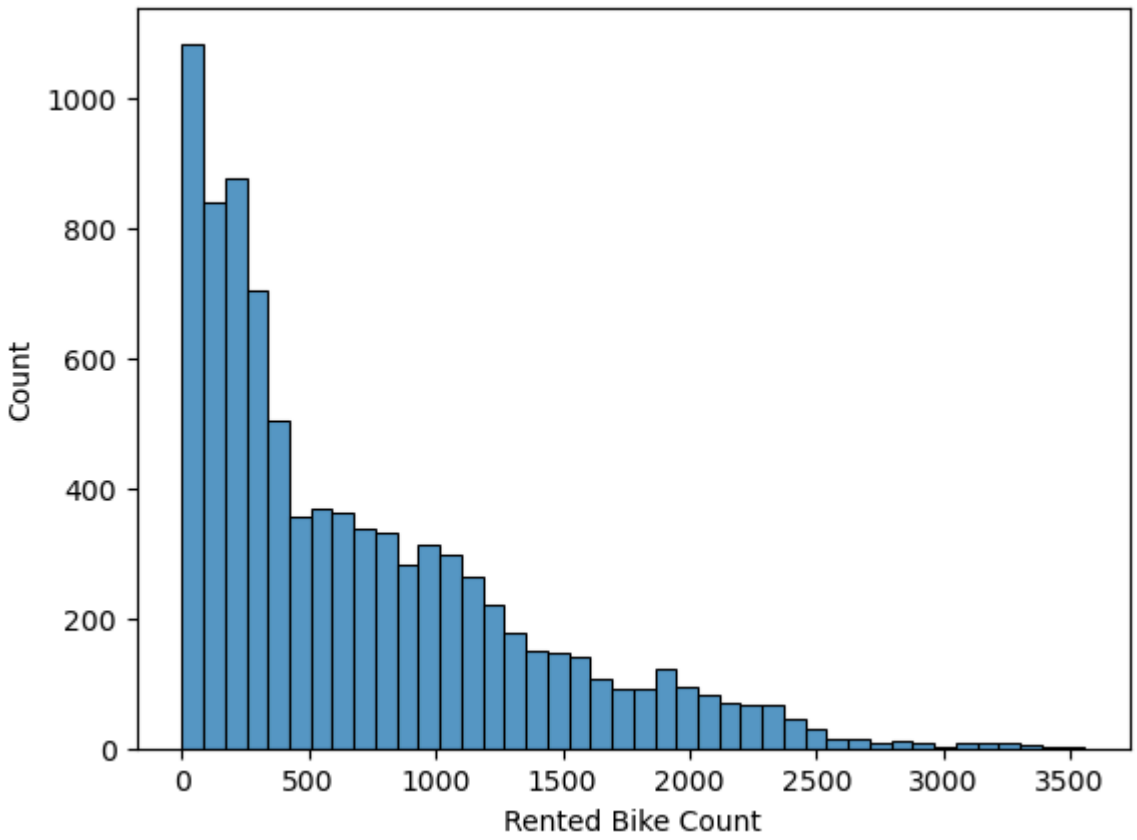
- profile report indicated skewed distributions for Wind Speed and Rented Bike Count - these are further examined for outliers.

In [13]:
```
sns.histplot(data=data, x='Wind speed (m/s)')
plt.show()
```



In [14]:
```
data = data[data['Wind speed (m/s)'] < 5]  # drop low frequency, extreme values (<5) as the outliers (can also check with IQR)
```

In [15]:
```
sns.histplot(data=data, x='Rented Bike Count')
plt.show()
```



In [16]:
```
data = data[data['Rented Bike Count'] < 2700]
```

Scaling

-- some regression models are influenced by the scale of input variables, which can distort coefficients/weights calculations and affect predictions - therefore appropriate scaling is required

- Decision trees do not require scaling on input variables, whilst for distance-based models such as kNN, scaling is important as input ranges influence distance calculations.
- Chosen to Standardization (instead of Normalization) as it is more robust.

Standardization (mean=0, std=1)

In [17]:
```
#std_scaler = StandardScaler()
```

In [18]:
```
# one-hot encoded features are not standardized

#std_num_vars = std_scaler.fit_transform(data.drop(['Spring','Summer','Winter', 'No Holiday'], axis=1))
```

In [19]:
```
#std_data = pd.DataFrame(std_num_vars, columns=data.columns[:-4])
#std_data['Spring']= data['Spring']
#std_data['Summer']= data['Summer']
#std_data['Winter']= data['Winter']
#std_data['No Holiday']= data['No Holiday']

#std_data.head(3)
```

Normalization (Min-max scaling)

In [20]:
```
# min_max = MinMaxScaler()
```

In [21]:
```
# scaling = min_max.fit_transform(data)
```

In [22]:
```
# scaled_data = pd.DataFrame(scaling, columns=data.columns)
# scaled_data.head()
```

Modeling

Regression Tree

In [23]:
```
X = data.drop(['Rented Bike Count'], axis=1)
y = data['Rented Bike Count']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

In [24]:
```
scaler = MinMaxScaler().fit(X_train)
```

In [25]:
```
scaled_Xtrain = scaler.transform(X_train)
scaled_Xtest = scaler.transform(X_test)
```

In [26]:
```
dtree = DecisionTreeRegressor(random_state=123, ccp_alpha=50).fit(scaled_Xtrain, y_train)
```

In [27]:
```
pred = dtree.predict(scaled_Xtest)
```

Test error

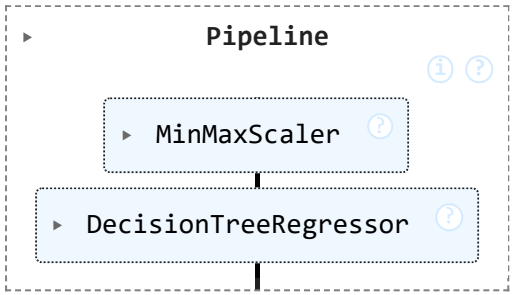
In [28]:
```
root_mean_squared_error(y_test, pred)
```

Out[28]:
```
388.96470398928875
```

Training error (cross validation)

In [29]:
```
pipeline = make_pipeline(MinMaxScaler(), DecisionTreeRegressor(ccp_alpha=50))
pipeline.fit(X_train, y_train)
```

Out[29]:



In [30]:
```
scores = -cross_val_score(pipeline, X_train, y_train, cv=10, scoring='neg_root_mean_squared_error')
scores
```

Out[30]:
```
array([345.09270216, 414.13100525, 436.11646348, 373.70544916,
       423.466443  , 392.5314411 , 393.10739577, 357.85619365,
       426.59300754, 402.43134957])
```

In [31]:
```
print("%0.2f RMSE with a standard deviation of %0.2f" % (scores.mean(), scores.std()))
```

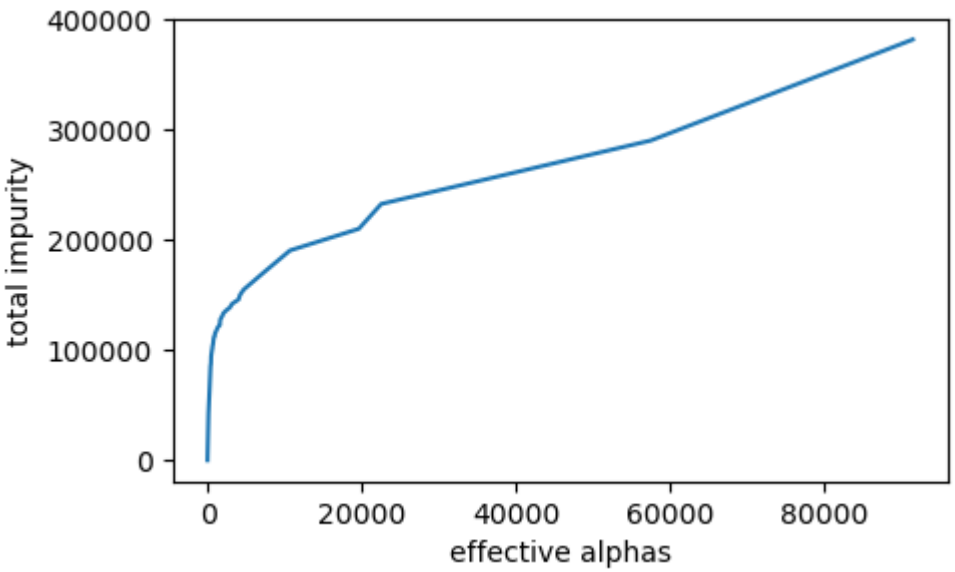
```
396.50 RMSE with a standard deviation of 28.69
```

Pruning

In [32]:
```
path = dtree.cost_complexity_pruning_path(scaled_Xtrain, y_train)
path
```

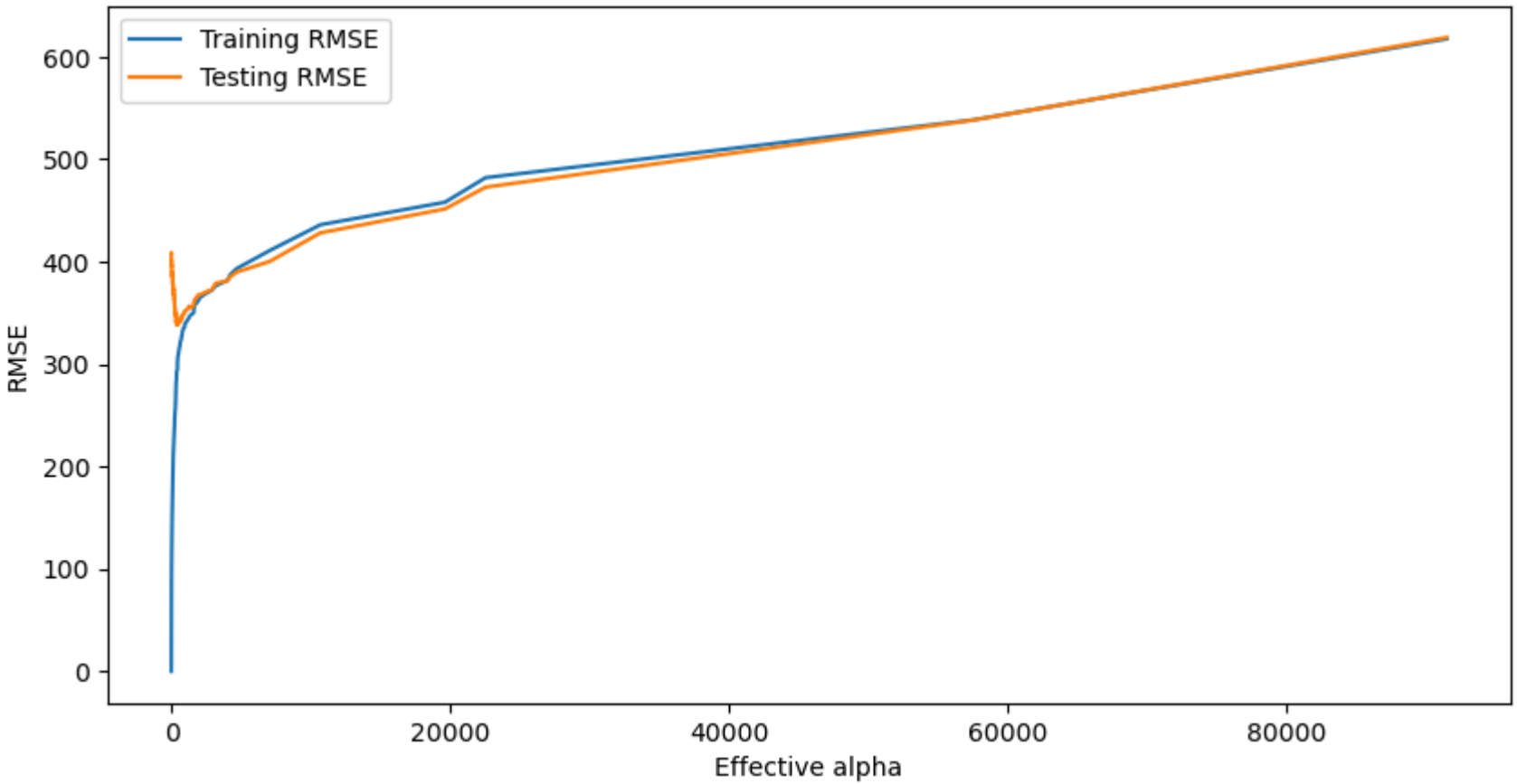
Out[32]:
```
{'ccp_alphas': array([0.00000000e+00, 8.25491167e-05, 8.25491167e-05, ...,
        2.25496415e+04, 5.75351514e+04, 9.15191145e+04]),
 'impurities': array([0.00000000e+00, 8.25491167e-05, 1.65098233e-04, ...,
        2.32392524e+05, 2.89927675e+05, 3.81446790e+05])}
```

In [33]:
```
alphas, impurities = path['ccp_alphas'], path['impurities']
plt.figure(figsize=(5,3))
plt.plot(alphas, impurities)
plt.xlabel('effective alphas')
plt.ylabel('total impurity')
plt.show()
```



In [34]:
```
train_acc, test_acc = [], []

for i in alphas:
    tree = DecisionTreeRegressor(random_state=123, ccp_alpha=i)
    tree.fit(scaled_Xtrain, y_train)
    y_train_pred = tree.predict(scaled_Xtrain)
    y_test_pred  = tree.predict(scaled_Xtest)

    train_acc.append(root_mean_squared_error(y_train, y_train_pred))
    test_acc.append(root_mean_squared_error(y_test, y_test_pred))
```

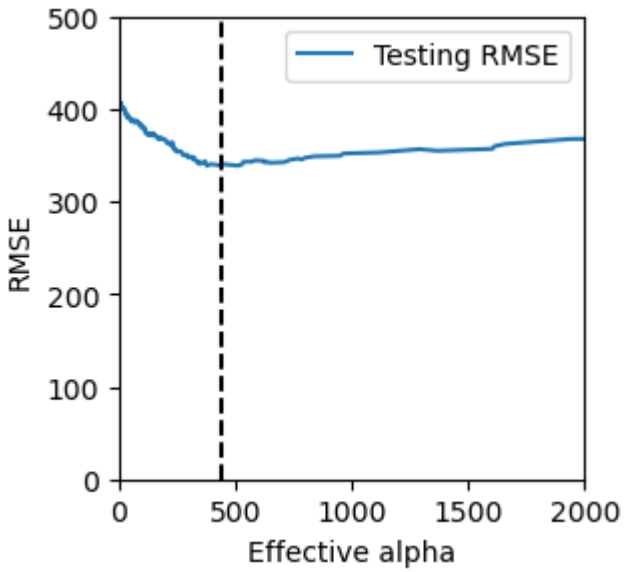
In [35]:
```
plt.figure(figsize=(10,5))
sns.lineplot(y=train_acc, x=alphas, label='Training RMSE')
sns.lineplot(y=test_acc, x=alphas, label='Testing RMSE')
plt.xlabel('Effective alpha')
plt.ylabel('RMSE')
plt.show()
```



In [36]:
```
print('%0.0f Alpha value for the lowest testing RMSE' % alphas[test_acc.index(min(test_acc))])
```

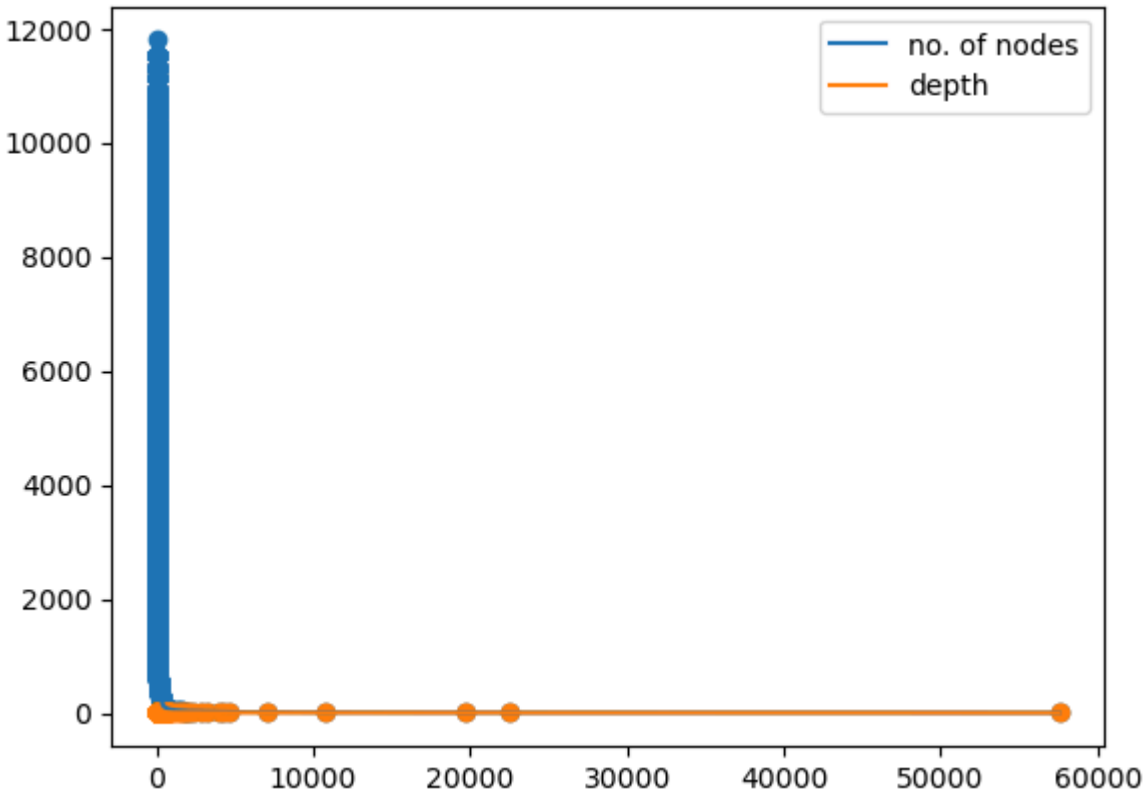
```
439 Alpha value for the lowest testing RMSE
```

In [37]:
```
# Testing accuracy only
plt.figure(figsize=(3,3))
sns.lineplot(y=test_acc, x=alphas, label='Testing RMSE')
plt.xlabel('Effective alpha')
plt.ylabel('RMSE')
plt.xlim(0,2000)
plt.ylim(0,500)
plt.axvline(439, linestyle='--', color='black')
plt.show()
```



In [38]:
```
trees = []

for alpha in alphas:
    tree = DecisionTreeRegressor(random_state=123, ccp_alpha=alpha)
    tree.fit(scaled_Xtrain, y_train)
    trees.append(tree)
```

In [39]:
```
trees = trees[:-1]
alphas = alphas[:-1]
nodes = [tree.tree_.node_count for tree in trees]
depth = [tree.tree_.max_depth for tree in trees]
plt.scatter(alphas, nodes)
plt.scatter(alphas, depth)
plt.plot(alphas, nodes, label='no. of nodes')
plt.plot(alphas, depth, label='depth')
plt.legend()
plt.show()
```



- A ccp alpha of approximately 439 corresponds to the lowest training and testing root mean squared errors (highest accuracy), alongside low tree depth and number of nodes - indicating a well-pruned, generalizable tree.

> Threfore, ccp_alpha = 439 is selected for the optimal decision tree

Therefore, final model:

Training cross validation

In [40]:
```
pruned_pipeline = make_pipeline(MinMaxScaler(), DecisionTreeRegressor(random_state=123, ccp_alpha=439))

pruned_pipeline.fit(scaled_Xtrain, y_train)
```

Out[40]:

```
▸ Pipeline
  ▸ MinMaxScaler
  ▸ DecisionTreeRegressor
```

In [41]:

```
# training cross validation on the pruned model

pruned_scores = -cross_val_score(pruned_pipeline, scaled_Xtrain, y_train, cv=10, scoring='neg_root_mean_squared_error')
```

In [42]:

```
print("%0.2f RMSE with a standard deviation of %0.2f" % (pruned_scores.mean(), pruned_scores.std()))
```

```
354.67 RMSE with a standard deviation of 29.12
```

Test error

In [43]:

```
# final model evaluation

opt_tree = DecisionTreeRegressor(random_state=123, ccp_alpha=439)
opt_tree.fit(scaled_Xtrain, y_train)
train_pred = opt_tree.predict(scaled_Xtrain)
test_pred = opt_tree.predict(scaled_Xtest)

print(f'Train RMSE: {root_mean_squared_error(y_train, train_pred)}')
print(f'Test RMSE: {root_mean_squared_error(y_test, test_pred)}')
```

```
Train RMSE: 294.3415189783267
Test RMSE: 339.20816669525726
```

- Train and test RMSE reduced after pruning with the optimal ccp_alpha, improving model accuracy and generalization.

> Pruned tree (Test RMSE = 339) < Unpruned tree (Test RMSE = 389)
>
> Pruned tree (Cross validation train RMSE = 355) < Unpruned tree (Cross validation train RMSE = 397)

---

kNN

In [44]:
```
knn = KNeighborsRegressor(n_neighbors=50)
```

In [45]:
```
knn.fit(scaled_Xtrain, y_train)
```

Out[45]:
```
KNeighborsRegressor
KNeighborsRegressor(n_neighbors=50)
```

Training cross validation error:

In [46]:
```
pipeline1 = make_pipeline(MinMaxScaler(), KNeighborsRegressor(n_neighbors=50))
pipeline1.fit(X_train, y_train)
```

Out[46]:
```
Pipeline
MinMaxScaler
KNeighborsRegressor
```

In [47]:
```
pipeline1.predict(X_test)
```

Out[47]:
```
array([1743.94,  943.88, 1251.82, ...,  242.48,  980.44,  262.06])
```

In [48]:
```
scores1 = -cross_val_score(pipeline1, X_train, y_train, cv=10, scoring='neg_root_mean_squared_error')
```

In [49]:
```
print('%0.2f Training cross validation RMSE with a standard deviation of %0.2f' % (scores1.mean(), scores1.std()))
```

```
390.34 Training cross validation RMSE with a standard deviation of 21.83
```

Test error

In [50]:
```
knn_pred1 = knn.predict(scaled_Xtest)
```

In [51]:
```
print(f'Test RMSE: {root_mean_squared_error(y_test, knn_pred1)}')
```

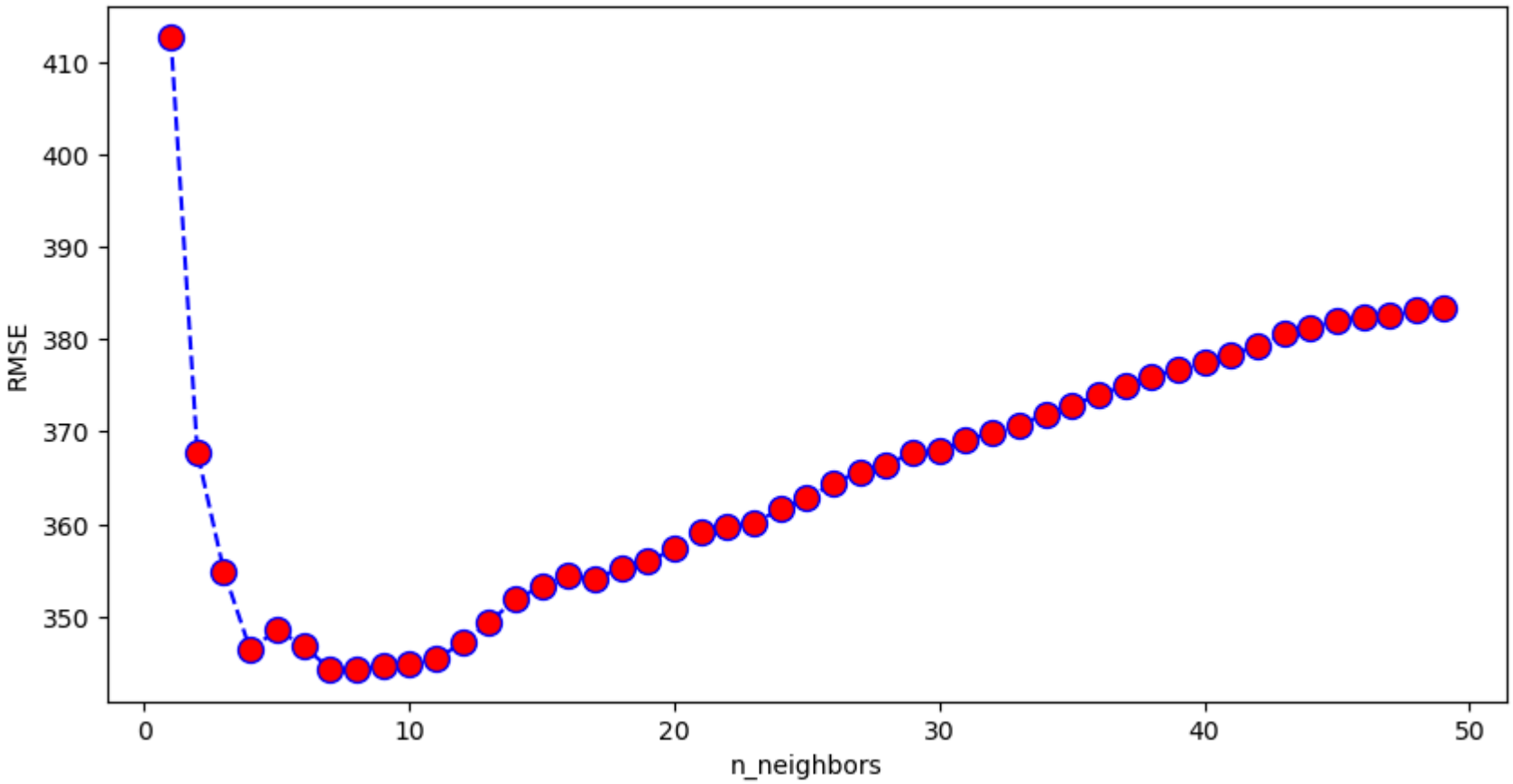
```
Test RMSE: 383.90311244845583
```

In [ ]:
```

```

Selecting the optimal no. of neighbors (k)

In [52]:
```
error = []

for i in range(1,50):
    knn = KNeighborsRegressor(n_neighbors=i)
    knn.fit(scaled_Xtrain, y_train)
    pred_i = knn.predict(scaled_Xtest)
    error.append(root_mean_squared_error(y_test, pred_i))
```

In [53]:
```
plt.figure(figsize=(10,5))
plt.plot(range(1,50), error, color='blue', linestyle='--', marker='o', markerfacecolor='red', markersize=10)
plt.xlabel('n_neighbors')
plt.ylabel('RMSE')
plt.show()
```



In [54]:
```
print(f'No. of neighbors for minimum RMSE: {error.index(min(error))}')
```

```
No. of neighbors for minimum RMSE: 7
```

Therefore, final model:

In [55]:
```
opt_knn = KNeighborsRegressor(n_neighbors=7)

opt_knn.fit(scaled_Xtrain, y_train)
```

Out[55]:
```
KNeighborsRegressor
KNeighborsRegressor(n_neighbors=7)
```

Training cross validation error:

In [56]:
```
pipeline2 = make_pipeline(MinMaxScaler(), KNeighborsRegressor(n_neighbors=7))
pipeline2.fit(X_train, y_train)
```

Out[56]:
```
Pipeline
MinMaxScaler
KNeighborsRegressor
```

In [57]:
```
pipeline2.predict(X_test)
```

Out[57]:
```
array([1750.57142857, 1079.57142857, 1216.85714286, ...,  341.        ,
        987.14285714,  243.        ])
```

In [58]:
```
scores2 = -cross_val_score(pipeline2, X_train, y_train, cv=10, scoring='neg_root_mean_squared_error')
```

In [61]:
```
print('%0.2f Training cross validation RMSE with a standard deviation of %0.2f' % (scores2.mean(), scores2.std()))
```

```
351.22 Training cross validation RMSE with a standard deviation of 24.24
```

In [ ]:

Test error

In [62]:
```
knn_pred2 = opt_knn.predict(scaled_Xtest)
```

In [63]:
```
print(f'Test RMSE: {root_mean_squared_error(y_test, knn_pred2)}')
```

```
Test RMSE: 344.32692166738514
```

- Training cross validation and test RMSE reduced after selecting the optimal number of neighbors, k=7 (from the initial guesstimate of 50), improving model accuracy and generalization:

> Training cross validation RMSE at k=7 (351) < Training cross validation RMSE at k=50 (390)
>
> Test RMSE at k=7 (344) < Test RMSE at k=50 (383)

Conclusion

- The Decision Tree generalizes marginally better than the kNN model, with a slighlty lower Test RMSE (339 < 344)
- kNN fits the training data marginally better than the Decision Tree, with a slightly lower cross validation training RMSE (351 < 355)

> The differences between the models are small, suggesting that both models perform similarly on this dataset.